

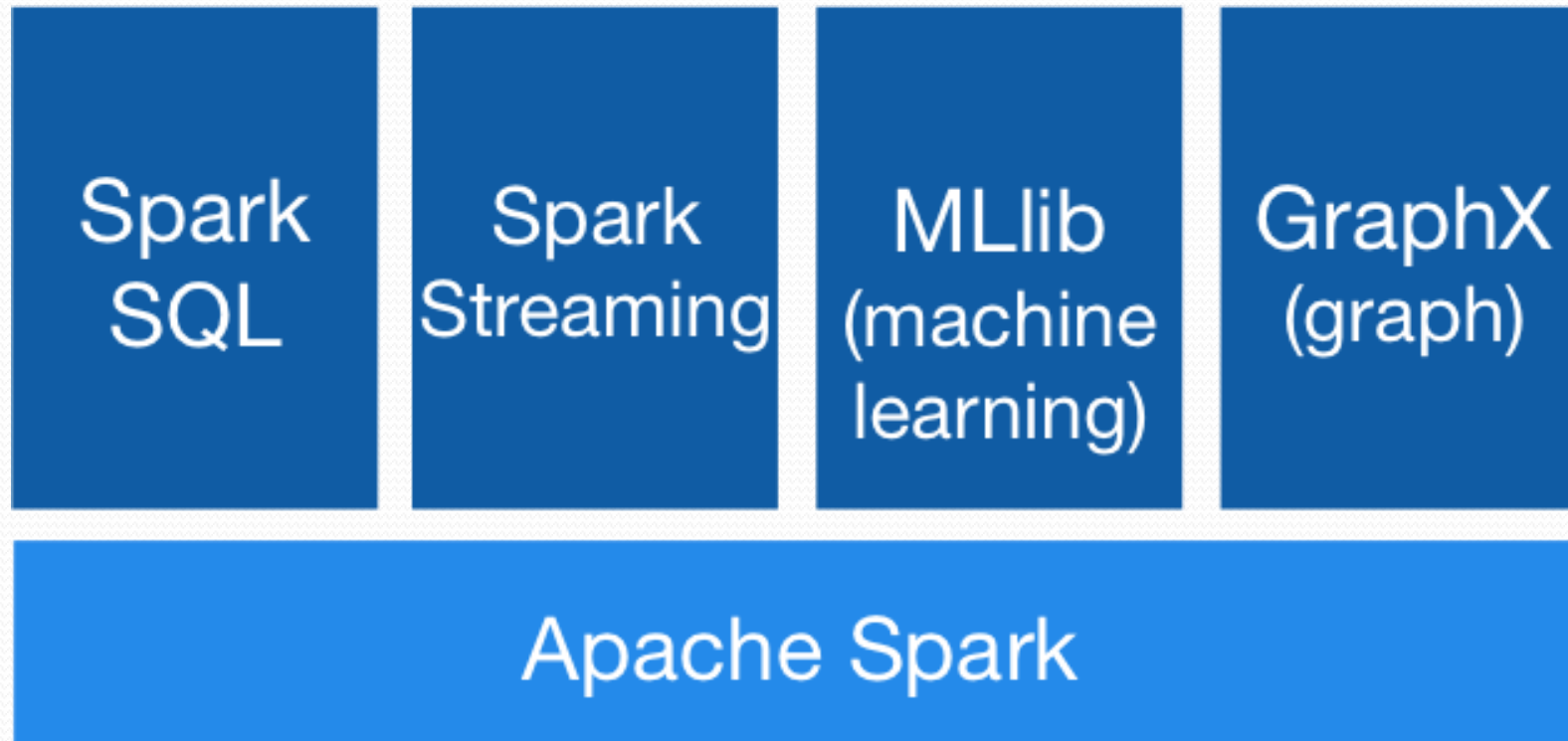
CS523 - Big Data Technology

Final Course Project

(Integrate Parts to Create a Whole)

**Knowing and Showing your Hidden Potential to Explore
Innovative Solutions for Large-scale Data Analysis**

Spark Ecosystem



Spark Structured Streaming

- Spark Structured Streaming is a scalable and fault-tolerant stream processing framework built on top of Apache Spark.
- It allows developers to process real-time data streams using the same high-level DataFrame and Dataset APIs (in Scala, Java or Python) that are used for batch processing.
- Rather than forcing developers to learn an entirely new programming model, Structured Streaming extends familiar Spark abstractions to continuous data, making stream processing more accessible and maintainable.
- Data ingestion can be done from many sources like Kafka, Flume, Amazon Kinesis or TCP sockets and processing can be done using complex algorithms that are expressed with high-level functions like map, reduce, joins, etc.
- Finally, processed data can be pushed out to filesystems, databases and live dashboards.

Spark Structured Streaming

- A key concept in Structured Streaming is the micro-batch execution model, where incoming data is processed in small, incremental batches at regular intervals.
- This approach provides strong consistency guarantees, including exactly-once processing semantics, while still enabling near-real-time analytics.
- Supports advanced operations such as windowed aggregations (allows the developer to specify a time frame to perform operations on the data that flows in that time window), stateful transformations (maintains a state based on data coming in a stream), joins between streaming and static data, and fault recovery using checkpoints.
- Transforms ingested events into insights using filtering, aggregation, anomaly detection, and moving averages.

Data stream

Unbounded Table



--	--	--

--	--	--

--	--	--

--	--	--

new data in the
data stream

=

new rows appended
to a unbounded table

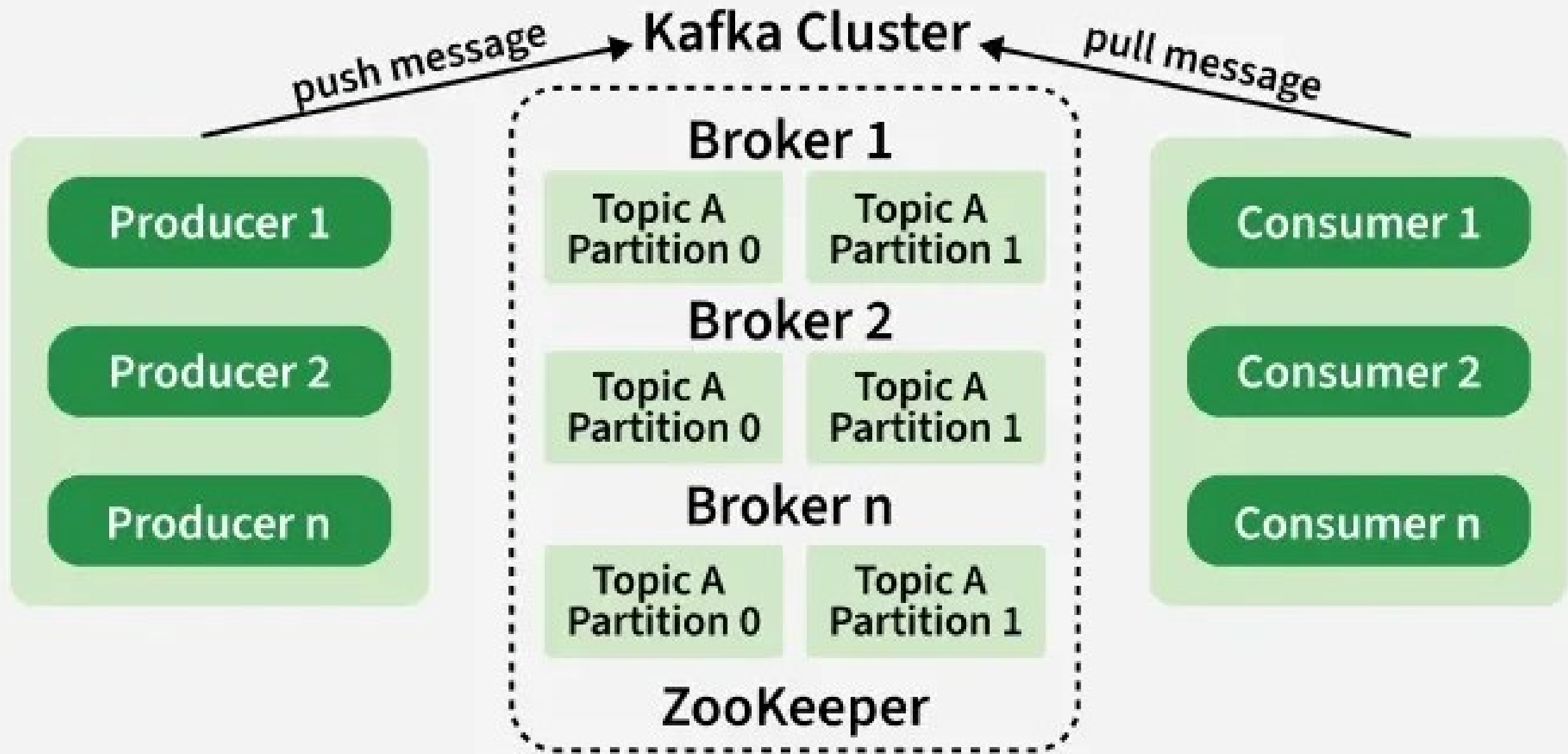
Data stream as an unbounded table

Spark SQL

- Spark SQL enables querying of structured and semi-structured data using SQL, DataFrame, and Dataset APIs.
- Utilizes catalyst optimization and whole-stage code generation for high performance analytics.
- Seamlessly integrates SQL analytics with both batch and streaming workloads for flexible data processing.
- Supports joining real-time streams with static datasets, enabling enriched insights and contextual analytics.

Apache Kafka

- Distributed event streaming platform designed to handle high-throughput, low-latency data feeds in real time.
- It enables applications to publish, subscribe to, store, and process continuous streams of records efficiently and reliably.
- Built around the concept of producers, topics, partitions, & consumers, allowing data to be written once and read many times by multiple downstream systems.
- This architecture makes Kafka well-suited for scenarios involving large-scale event ingestion, log aggregation, and real-time analytics pipelines.
- Kafka decouples data producers from processors, improving scalability, flexibility, and system resilience.
- Kafka ensures fault tolerance and message durability through partition replication across brokers.



Project Overview

1) End-to-End Big Data Pipeline

- Project involves building a full pipeline integrating real-time data ingestion, processing, storage, and visualization.

2) Real-Time Streaming Data

- Students work with high-volume, real-time streaming data from public sources to simulate practical scenarios.

3) Distributed Computing and Storage

- Processing uses distributed computing and stores results in NoSQL or distributed data warehouses for scalability.

4) Visualization and Insights

- Results are presented via live dashboards that update frequently to show analytics insights clearly.

Architecture Overview

- **Data Ingestion with Kafka**
 - Data is ingested from real-time sources into Kafka, providing scalable, durable buffering for downstream processing.
- **Distributed Processing with Spark**
 - Spark Structured Streaming performs distributed filtering, aggregation, joins, and anomaly detection on the ingested data.
- **Persistent Storage Layer**
 - Processed data is stored in persistent layers like HBase or Hive to support various access and query patterns.
- **Visualization and Insight Delivery**
 - Dashboards connect to storage layers, presenting real-time or near real-time insights to end users.

Project Details

Project Parts	Points	Short Description of the Project Part
1	3	Data Ingestion using Kafka
2	3	Distributed Processing using Spark Structured Streaming
3	2	Saving Processed data to HBase or Hive
4	2	Dynamic Dashboards to visualize insights and results
5	2 Bonus points	Use Spark SQL to join streaming data with static datasets
Total	10	10% of the total course grade

Project Parts Details

- **Part 1.** [3] **Real-Time Data Ingestion (Apache Kafka)**

- ✓ Identify a free, high-volume, real-time data source (e.g., Binance WebSocket for cryptocurrency trades, Wikimedia Recent Changes stream, Meetup.com RSVP stream, or a public IoT sensor feed).
- ✓ Write a producer script (Java or Python) that connects to this API and streams the raw data into an Apache Kafka topic.

- **Part 2.** [3] **Distributed Processing (Spark Structured Streaming)**

- ✓ Write an Apache Spark Structured Streaming application that subscribes to your Kafka topic.
- ✓ Your Spark application must perform meaningful transformations or aggregations on the data in real time (e.g., windowed aggregations, filtering anomalies, joining with static reference data, or calculating moving averages).

Project Parts Details contd...

- **Part 3.** [2] **Persistent Storage (HBase or Hive)**

- ✓ Sink your processed DataFrames from Spark into a persistent storage layer.
- ✓ Option A (HBase): Best for storing aggregated data that requires rapid, real-time key-value lookups for your dashboard.
- ✓ Option B (Hive): Best for appending structured, batch-oriented summaries of your streaming windows.

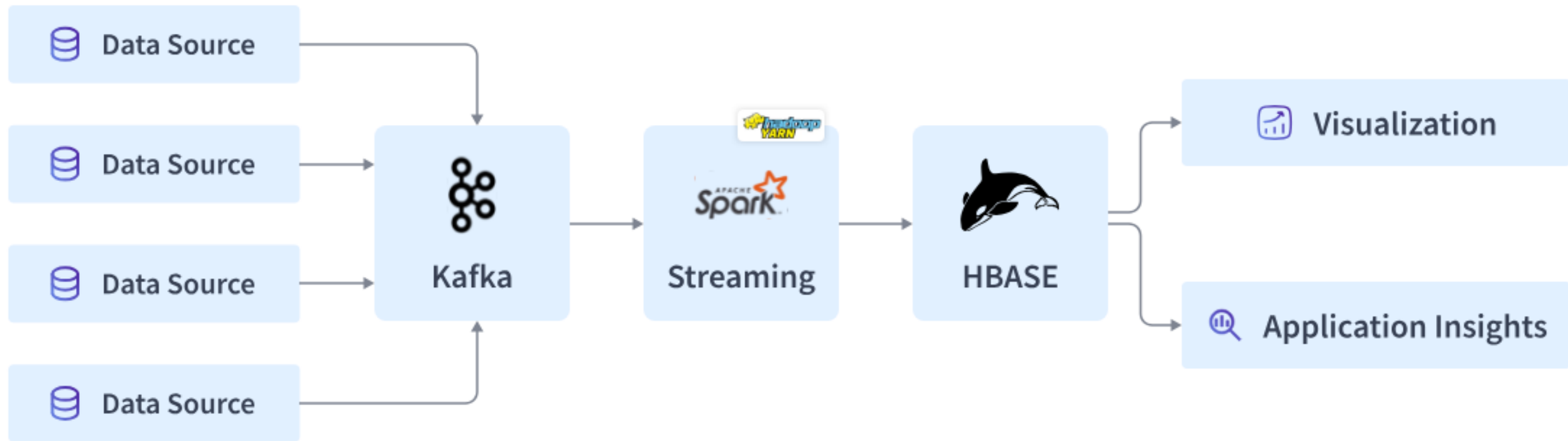
- **Part 4.** [2] **Visualization & Dashboarding**

- ✓ Connect your persistent storage layer to a visualization tool to create a live (or frequently updating) dashboard of your insights.
- ✓ You may use the ELK Stack (Elasticsearch/Kibana), Streamlit, Grafana, Tableau, or a custom web application.

Project Parts Details contd...

- **Part 5.** [2 bonus points] **Data Enrichment with Spark SQL**
 - ✓ To earn bonus points on your final project grade, use Spark SQL to join your live streaming data with a static dataset stored in HDFS.
 - ✓ Example: If your stream contains raw transaction IDs, load a static CSV of product details from HDFS and join them together in your Spark code to enrich the data before writing it to your database.

Sample Data Flow Diagram



Static Public Datasets

If you are unable to locate real-time Streaming data sources, then look at the following sites and find your favorite static dataset to simulate streaming and do further analysis.

- [Amazon Web Services](#)
- [UCI Machine Learning Repository](#)
- [Kaggle](#)
- [US Government's Open Data](#)

Project Deliverables

1. Source Code Repository Submission

- ❑ Provide a link to a public GitHub repository containing all your project files, including your Kafka producer scripts, Spark application code, database schema definitions, and visualization configurations.
- ❑ Include a README.md file with clear instructions on how to start your pipeline.

2. Video Demonstration ([OBS Studio](#), [Loom](#) or Streams could be used for recording)

- ❑ Record a video presentation (max 20 mins) showcasing your project. Upload the video to Microsoft Streams and submit the link.
- ❑ Show the data flowing from the source API into Kafka, through Spark, and finally updating your visualization dashboard in real time.
- ❑ All team members are required to appear in the video. Each member must be visibly present and actively participate in the demo presentation to get points.